

DOI:10.16644/j.cnki.cn33-1094/tp.2026.02.010

基于LangChain的检索增强生成知识库问答系统设计与实现

陈文丽¹, 张洪波¹, 靳德政²

(1. 郑州西亚斯学院数字技术产业学院, 河南 郑州 451150; 2. 河南理工大学鹤壁工程技术学院)

摘要: 当前,大语言模型在智能问答领域已广泛应用,并在通用场景下表现出较强的语言理解与生成能力。然而,在金融、法律等特定行业应用中,受限于训练数据,大语言模型往往难以准确回答专业问题,甚至产生“幻觉”现象。针对上述问题,本文引入检索增强生成(Retrieval-Augmented Generation, RAG)技术,基于主流的大语言模型应用开发框架LangChain开发了一套面向特定领域的知识库问答系统,该系统通过构建领域知识向量库、采用提示词拼接方法,有效提升大模型在特定领域的问答效果。实验结果表明,使用RAG技术后,系统在回答特定领域问题的准确性与相关性显著提升,验证了该系统的可行性。

关键词: 大语言模型; 检索增强生成; LangChain; 问答系统

中图分类号: TP391

文献标识码: A

文章编号: 1006-8228(2026)02-51-06

Design and Implementation of a Retrieval-Augmented Generation Knowledge Base Question Answering System Based on LangChain

Chen Wenli¹, Zhang Hongbo¹, Jin Dezheng²

(1. Digital Technology School of Sias University, Zhengzhou, Henan 451150, China;

2. Hebi Institute of Engineering and Technology, Henan Polytechnic University)

Abstract: Currently, large language models are widely used in intelligent question answering, which demonstrate strong language understanding and generation capabilities in general scenarios. However, in specific domain applications such as finance and law, limitations in training data often hinder accurate answers to specialized questions, sometimes even leading to "hallucination" responses. To address these issues, this paper introduces Retrieval-Augmented Generation (RAG) technology. Based on the mainstream large language model framework LangChain, a domain-specific knowledge base question answering system is developed. This system effectively improves the question answering performance of large models in specific domains by constructing a domain knowledge vector library and employing a method of concatenating prompt words. Experimental results show that using RAG technology significantly improves the accuracy and relevance of the system in answering domain-specific questions, validating the feasibility of the system.

Keywords: Large Language Model; Retrieval-Augmented Generation; LangChain; Question Answering System

0 引言

近年来,大语言模型(Large Language Model, LLM)在自然语言领域取得了突破性进展。以 ChatGPT、DeepSeek、通义千问等为代表的大语言模型,其参数规模已达到千亿级别^[1],在信息检索、智能问答等领域展现出较强的能力。艾媒咨询(iiMedia Research)数据显示,2024年中国大模型市场规模已达到294.16亿元人民币,大模型正逐步成为新一代信息技术的重要

技术基础。

以 GPT-3^[2]为例,其训练资料包含网络爬虫获取的内容、书籍资料、维基百科等互联网公开资料,这使得 GPT-3 在公开资料的领域中回答问题效果较好。但是大模型在面对专业领域的问题时,经常无法准确回答,甚至会虚构回答,即所谓的大模型“幻觉”问题。在一些特定行业中,如金融、医疗、法律等领域,难以直接应用大模型。

收稿日期: 2026-01-03

作者简介: 陈文丽(1997—),女,河北张家口人,硕士,助教,主要研究方向:网络安全。

为缓解上述问题,OpenAI支持大模型微调(fine-tuning)^[3],即在预训练模型上使用符合特定领域的少量数据继续训练模型。在训练过程中,大模型的参数将被更新以适应特定领域的问答需求。但是微调模型需要对预训练模型进行训练,对时间与算力要求较高。

检索增强生成(RAG)技术为解决该问题提供了新的思路,与微调技术不同,RAG不对模型参数进行调整,而是从用户提供的外部知识库中检索并整合相关信息,然后将问题和知识库中相关内容一并提供给大模型,从而生成质量更好的回答。由于无需对模型进行微调,该技术相较于微调技术更经济,逐渐成为特定领域问答系统的主流方案^[4]。

基于此,本文依托LangChain^[5]框架,设计并实现了一种面向特定领域的RAG知识库问答系统。通过对知识库内容进行向量化存储,结合检索、提示词构建等技术手段,实现了对特定领域问题精确回答的效果。实验结果表明,该系统可以显著提升特定领域问答系统的准确性和相关性,为大语言模型在实际场景中的落地提供了一种可行的工程实现方案。

1 相关技术

1.1 大模型

大语言模型是基于深度学习的大规模预训练语言模型,属于自然语言处理技术,一般使用Transformer架构^[6],通过在大规模的语料上进行训练,学习语言的统计规律与语义表示能力,从而达到文本理解、生成等能力。2022年11月30日,OpenAI发布了ChatGPT^[7],该模型允许用户使用自然语言对话的形式进行交互,可以实现翻译、聊天对话、问答等任务。近年来,大语言模型技术在国内也得到了快速发展,国内科研机构与互联网企业相继开展大语言模型相关研究与落地应用,目前较为流行的模型有:DeepSeek、通义千问、豆包等。

然而,大语言模型的知识主要来源于训练阶段的静态数据,对较新的数据及特定领域的知识更新能力有限,在回答这些问题时,容易出现回答不准确或产生幻觉的问题,这导致了大语言模型在特定领域难以应用。本文将研究通过RAG技术引入外部知识库,以增强大语言模型在特定领域回答问题的能力。

1.2 LangChain框架

LangChain是一个面向大语言模型应用开发的框

架,采用模块化的方式与链式设计思想,将提示词构建、工具调用、模型调用等过程有机组合,提高了大模型应用的可拓展性。在知识库问答场景中,LangChain提供了文档导入、文本切分、向量化、向量数据库、检索器等组件,可以通过上述组件快速生成RAG应用。

在知识库问答领域,除LangChain以外,还存在Coze、Dify、AnythingLLM等可视化平台,这类平台可以通过图形化界面配置知识库问答,在快速原型开发方面有一定优势。然而,由于其使用图形化界面,内在封装程度较高,在检索流程、提示词构造等方面可拓展性有限。在RAG应用方面,具体对比结果如表1所示,相比之下,LangChain采用模块化与链式设计思想,提供了灵活的工程控制能力,能够对模型调用、文本向量化、检索策略以及上下文构建进行更精细化控制,更适合实际企业场景应用落地。因此,本文选用LangChain框架作为RAG应用实现的基础。

表1 LangChain与其他平台对比

对比维度	LangChain	Coze/Dify/AnythingLLM
技术定位	代码级开发框架	可视化平台
灵活性	高	低
可扩展性	强	中等
实验可控性	强	较弱
开发难度	中	低

1.3 检索增强生成(RAG)技术

检索增强生成是一种将信息检索与文本生成相结合的技术框架,其核心思想是在用户将提示词发送至大语言模型前,先从外部知识库中检索与用户问题相关的文本信息,并将检索结果作为上下文输入至大语言模型中,从而使大语言模型获取问题相关内容并生成更加准确和可靠的回答。

RAG典型流程包括文档文本向量化、语义检索以及结果融合生成等步骤。通过将特定领域知识以向量形式存储在向量数据库中,RAG系统能够在用户提问时与向量数据库中的向量进行向量相似度匹配,并获取与问题最相关的知识片段。相比于仅依赖通用知识的大语言模型,RAG技术能够有效缓解模型知识不足和幻觉问题,在特定领域知识问答任务中具有较好的应用效果。

如图1所示,RAG技术应用的工作流程如下:

(1) 本地知识库向量化存储。用户的本地文档,如Word、PDF、txt等可转换为文本的资料,首先使用非结构化文档加载器(Unstructured Loader)转换为文

本,使用文本切分器根据切分规则切分为文本片段,再通过 Embedding 模型计算文本片段的向量并存储至向量数据库中,此时便构建好了可检索的知识库。

(2) 语义检索与构建 Prompt 阶段。在此阶段,用户问题使用 Embedding 模型向量化为查询向量,然后使用余弦相似度与向量数据库中的向量进行匹配,检索最相似的几个文本片段,然后使用提示词模板构建为新的提示词。其中,对于两个 n 维向量 $A = (a_1, a_2, \dots, a_n)$ 和 $B = (b_1, b_2, \dots, b_n)$, 余弦相似度 (Cosine Similarity) 计算公式为:

$$\text{Cosine Similarity}(A, B) = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2 \sum_{i=1}^n b_i^2}} \quad (1)$$

其中, a_i 和 b_i 分别表示向量 A 和 B 在第 i 个维度的分量, Σ 表示对所有维度求和,分母为两个向量的欧几里得范数(即向量长度),在实际文本向量匹配中,余弦相似度取值范围为 0 到 1,值越接近 1 表示两个向量越相似,即用户问题与向量数据库中的内容匹配度越高。

(3) 将构建完成的提示词发送给大模型,大模型根据用户问题、相关文本片段以及预置的提示词回答问题。

通过上述工作流程可以发现,RAG 应用的回答质量好坏与以下因素相关:本地知识库、Embedding 模型、文本切分规则、检索的文本片段数量、预置提示词、大模型等。

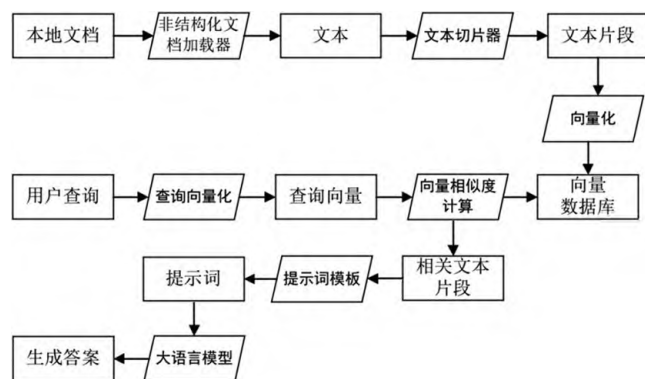


图1 RAG 技术应用工作流程

2 基于 LangChain 的 RAG 系统实现

2.1 系统架构

如图2所示,自下到上本系统架构分为五个层次:

(1) 数据存储层。数据存储层包含特定领域知识库、向量数据库,该层负责管理结构化与非结构化数据,保存用户的知识库文档与文档切片生成的向量。

(2) 检索服务层。检索服务层包括文档加载器、文本分块器、向量嵌入模型以及相似度检索模块。该层支持了 RAG 技术中核心的语义检索,为智能推理层提供了知识检索支持。

(3) 智能推理层。智能推理层包含 Agent、系统提示词管理、工具编排以及大语言模型。该层负责自然语言理解、使用大语言模型推理决策并调用工具,以实现复杂的业务逻辑自动化处理。

(4) 应用服务层。应用服务层包括 FastAPI 服务,该服务使用 GET、POST 请求方式对外暴露接口。该层负责请求处理,包括传入的参数处理、业务逻辑调度等。

(5) 用户交互层。用户交互层即可视化界面,用户可以直接交互操作,也可以使用 Open WebUI、ChatBox 等第三方客户端接入系统服务。



图2 系统架构图

2.2 知识库向量化方法与实现

为体现真实使用场景,本文使用成都信息工程大学学生手册(PDF版)、相关规章制度的图片类文档作为知识库文档。其中,学生手册在官网下载,该文档包含学校概况、政策规章、奖惩管理、教学管理、资助政策、住宿管理等内容,共 406 页。本文使用上述资料构建了一个学生手册问答助手。

在本文 RAG 系统向量化数据库实现中,首先使用 PyPDFLoader 加载学生手册文档(PDF),该方法会将 PDF 文档进行切片处理,每一页切成一个 Document 对象,该对象包含了文本及相关的元数据,示例: `page_content= "Hello, world!", metadata= {"source": "https://example.com"}`。当 PDF 文档切分成 Docu-

ment 对象列表后,使用 text_splitter 将文档切分成文本片段 (Chunk), 这里切分可以设置一定的规则: CHUNK_SIZE、CHUNK_OVERLAP, 其中 CHUNK_SIZE 指的是文本片段的大小, CHUNK_OVERLAP 指的是每块文本之间重叠的字符数。在文本切分过程中, 本文对不同的切分粒度进行了对比实验, 实验结果表明, 对于本文使用的知识库内容, CHUNK_SIZE 设置为 1000 字符、CHUNK_OVERLAP 设置为 200 字符时, 能够在保证语义完整性的同时, 避免上下文信息割裂, 从而可以提升检索与后续生成内容的效果。相关规章制度图片使用 Tesseract OCR 工具识别为文本后, 再按照上述相同规则切分为文本片段。

当切分完成后, 可以使用 Embedding 模型将文档的片段向量化后存入向量数据库。本文使用 Chroma 向量数据库, 其支持高效的向量相似度搜索, 兼容性较好, 可以与 LangChain 无缝集成。Embedding 模型使用 text-embedding-3-large, 其支持 3072 维向量输出, 可以精细地捕捉文本的语义特征, 理解长文本与复杂语义内容, 并且与 Chroma 向量数据库兼容, 以共同实现高效的语义检索。

经过对文档、图片处理与向量化存储后, 后续检索可以同时访问文档与图片的内容, 实现多模态统一检索功能。为了进一步提高特定领域知识的检索精度, 本文 RAG 系统在检索阶段引入了混合检索策略: 在基于语义相似度的基础上, 引入领域权重, 对知识库中不同类型的文档片段进行加权处理。例如政策条款、奖惩办法等关键内容提高权重, 而目录、表格等非核心信息降低权重。在获取到相关候选片段后, 系统结合余弦相似度与领域权重重新计算排序, 最终选择最相关的片段作为检索结果, 从而更准确地匹配用户问题与具体知识。

2.3 Agent 工程实现

在本系统中, 检索器的调用通过 LangChain 的 Agent 实现, Agent 的创建包含三个核心组件: 大语言模型 (LLM)、工具 (tools)、提示词 (prompt)。为了兼顾较好的推理及回答效果与合理的计算成本, 本系统使用了 gpt-4o-mini 模型。Agent 负责管理工具与调用逻辑, 通过将工具定义注入提示词, 使大模型理解在何种条件下调用特定工具, 然后通过结构化输出调用请求 (tool_call), 指示具体的工具调用请求。在本系统中, 工具的定义如下:

```
@tool(response_format="content_and_artifact")
```

```
def retrieve_context(query: str):
```

```
    """检索文档上下文用于回答问题"""
```

```
    # 工具实现...
```

在 Agent 的提示词中, 本文限定了 Agent 只能使用 retrieve_context 工具检索信息并回答问题, 完整提示词如下:

"你是成都信息工程大学学生手册的智能助理。你的任务是回答用户问题。"

"请严格遵循以下规则:"

"1. 只能使用 retrieve_context 工具提供的文档信息回答问题, 不允许凭个人记忆回答。\\n"

"2. 如果文档中没有相关信息, 请明确回复: 未找到相关信息。\\n"

"3. 回答尽量简洁明了, 每句话尽量引用文档来源。\\n"

"4. 文档引用格式: 引用自: <doc_name>\\n"

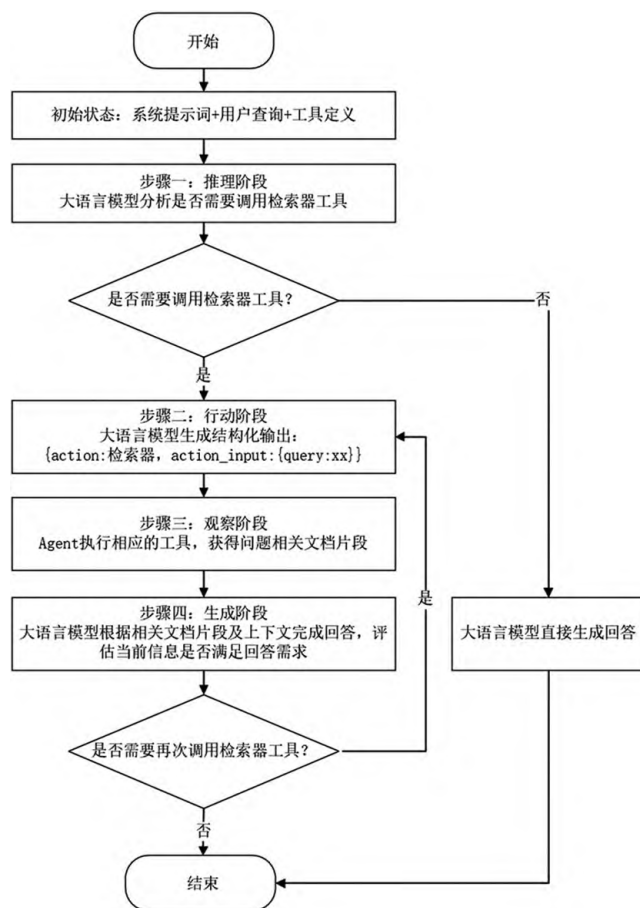


图3 检索增强生成(RAG)的问答流程示意图

结合上述工具与提示词, 应用到 LangChain 的 ReAct (Reasoning + Acting) 框架, 从用户输入到回答的流程如图3所示。在此过程中, 用户提供的问题先与系统提示词、工具定义同发送给大模型进行推理, 然后大模型推理决定是否调用工具。若需要工具则

生成调用工具的结构化输出 tool_call, 包含调用工具名称、参数等信息。当 Agent 收到大模型调用工具的 tool_call 结构化输出后, 首先使用 Embedding 模型向量化经过大模型优化的用户问题, 然后使用检索器工具查询向量数据库中最相似的 top-k 条文档片段, Agent 将获取的文档片段与历史上下文一并发给大模型, 大模型思考并给出回答, 若大模型认为还需要再次调用检索器, 则继续重复上述过程。为了提升 Agent 的响应效率, 同时兼顾检索文本片段的效果, 本文对 Agent 调用检索器的次数进行了限制, 经过多次实验对比, 最终将 Agent 调用检索器的最大尝试次数设置为 3 次。图 4 展示了使用 LangSmith 追踪的工作流程。



图 4 使用 LangSmith 追踪工作流程截图

3 实验对比

3.1 实验环境

本系统使用的硬件环境为搭载 Ubuntu 20.04 操作系统的服务器, CPU 为 8 核, 内存为 8 GB, 无独立显卡。使用 Python 版本为 3.10, 所采用的大模型为 gpt-4o-mini, Embedding 模型为 text-embedding-3-large, 模型通过第三方 API 转发服务接入。系统使用 FastAPI 暴露接口, 包含 /v1/models 与 /v1/chat/completions, 其中 /v1/models 接口返回系统可用模型, 以兼容 Open WebUI 和 ChatBox 等大模型主流的前端工具, /v1/chat/completions 是问答接口, 遵循 OpenAI 接口规范, 支持流式响应与非流式响应。本地模型通过 Ollama 部署, 根据硬件性能, 选用了适配硬件性能的目前主流的模型, 大模型使用 Qwen2.5-3B-Instruct, Embedding 模型使用 nomic-embed-text:latest。

在实验测试阶段, 本文使用 ChatBox 接入系统, 具体方式如下:

(1) 启动 RAG 系统, 获取访问链接。

(2) 在 ChatBox 设置界面中, 添加模型提供方, 选择 OpenAI (Responses), 在 API 主机中填入步骤 (1) 中获取的访问链接, 点击“获取模型”加载 RAG 系统中配置的模型。

(3) 返回 ChatBox 主页, 点击“新对话”, 选择刚获取的模型, 即可开始与知识库问答交互操作 (见图 5)。



图 5 使用 ChatBox 与 RAG 系统问答交互

3.2 对比实验分析

为验证本系统搭建的 RAG 系统效果, 本文以《成都信息工程大学学生手册》、相关规章制度图片为知识来源, 从学生视角设计了 20 条典型的问题, 包含教学管理类、资助政策类、奖惩管理类、政策规章类与住宿管理类, 每个分类各 4 条问题, 同时设计了问题回复精准度评分标准 (见表 2)。本文使用 RAG 系统、RAG 系统 (本地模型)、ChatGPT 在线版对上述问题进行提问并记录回答结果, 如表 3 所示。

表 2 RAG 系统回答效果评分标准

评分标准	得分
无结果	0
有结果, 不理解问题, 文不对题	1
有结果, 理解问题, 但是信息不正确	2
有结果, 理解问题, 但是含有较多的不相关内容	3
有结果, 理解问题, 但是含有较少的不相关内容	4
有结果, 理解问题, 精准回答问题, 言简意赅	5

由表 3 的实验数据可见, 测试结果中, 基于云端大模型的 RAG 系统回答效果最好, ChatGPT 在线版其次, RAG 系统 (本地模型) 回答效果相对较差。其主要原因在于, RAG 系统使用了包含学生手册的向量数据库, 模型可以根据检索到的相关文档内容回答; 相比之下, ChatGPT 在线版没有接入该知识库, 只能依赖通用模型知识或联网搜索。

表3 实验结果对比

问题分组	RAG(云端)	ChatGPT在线版	RAG(本地模型)
教学管理类	18	14	5
资助政策类	19	11	9
奖惩管理类	17	12	7
政策规章类	18	15	10
住宿管理类	16	11	6
总分	88	63	37

尽管基于本地模型的RAG系统包含学生手册的向量数据库,但是回答质量仍然偏低,这是因为,本地的大模型不能很好地理解系统提示词,难以稳定产生调用工具的tool_call结构化输出;此外,本地向量数据库的向量维度较低,相比之下,text-embedding-3-large模型生成的向量维度为3072,能够对长文本及复杂语义进行精细表征,而本地模型RAG系统使用的Embedding模型nomic-embed-text的向量维度只有768,对语义信息表达能力相对不足,进一步影响了检索与回答效果。

4 结论

本文设计并实现了一个基于LangChain的检索增强生成知识库问答系统,通过引入外部知识库,有效增强大语言模型在特定领域的问答能力。实验结果表明,该系统在回答特定领域问题时,准确性与相关性均优于未引入外部知识库的大语言模型。相较于直接训练大模型或微调大模型,所提出的方法在时间

成本与硬件成本上具有明显优势,能够以低成本实现领域知识增强,具有较好的通用性,可以有效应用于其他特定领域,具有一定的工程实践价值。

参考文献(References):

- [1] 秦小林,古徐,李弟斌,等.大语言模型综述与展望[J].计算机应用,2025,45(3):685-696.
- [2] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc.,2020,33:1877-1901.
- [3] 张钦彤,王昱超,王鹤羲,等.大语言模型微调技术的研究综述[J].计算机工程与应用,2024,60(17):17-33.
- [4] 刘雪颖,云静,李博,等.基于大型语言模型的检索增强生成综述[J].计算机工程与应用,2025,61(13):1-25.
- [5] TOPSAKAL O, AKINCI T C. Creating large language model applications utilizing LangChain: A primer on developing LLM apps fast[C]//International Conference on Applied Engineering and Natural Sciences. Konya, Turkey: All Sciences Proceedings,2023,1(1):1050-1056.
- [6] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc.,2017,30:5998-6008.
- [7] NAZIR A, WANG Z. A comprehensive survey of ChatGPT: advancements, applications, prospects, and challenges[J]. Meta-Radiology,2023,1(2):100022. 

(上接第50页)

征后,模型准确率达96.75%,AUC值达0.9845,且单条数据的检测耗时仅5.7 ms,满足实时部署需求,表现出较强的鲁棒性与泛化能力。然而,本研究仍存在以下局限性:例如,模型对动态域名生成、短链跳转等新型钓鱼手法的适应性有限;模型训练依赖高质量标注数据,难以应对零日钓鱼攻击等场景。后续研究将引入页面DOM(Document Object Model,文档对象模型)结构分析、用户行为序列挖掘等深度动态特征,提升特征提取的全面性和有效性;探索半监督学习与自监督学习方法,降低模型对标注数据的依赖;进一步提升模型对未知钓鱼攻击的检测能力与泛化能力。

参考文献(References):

- [1] 曹玖新,董丹,毛波,等.基于URL特征的Phishing检测方法[J].东南大学学报(英文版),2013,29(2):134-138.

- [2] 顾晓清,王洪元,倪彤光,等.基于贝叶斯和支持向量机的钓鱼网站检测方法[J].计算机工程与应用,2015(4):87-90,95.
- [3] 莫玉力.基于SVM和神经网络的URL安全检测[D].北京:北京邮电大学,2016.
- [4] 李蓓.基于DPI数据的恶意URL检测方法研究[D].北京:北京邮电大学,2019.
- [5] PRAKASH P, KUMAR M, KOMPELLA R R, et al. PhishNet: Predictive blacklisting to detect phishing attacks[C]//Proceedings of IEEE INFOCOM 2010. San Diego, CA, USA:IEEE Computer Society,2010:1-5.
- [6] VANHOENSHOVEN F, NÁPOLES G, FALCON R, et al. Detecting malicious URLs using machine learning techniques[C]//2016 IEEE Symposium Series on Computational Intelligence,Athens,Greece:IEEE,2017:1-8.
- [7] 毛红梅,张胜.随机森林算法在钓鱼网站检测中的应用[J].电脑知识与技术,2023,19(23):90-92. 